Rockchip RT-Thread I2C开发向导

文件标识：RK-KF-YF-113

发布版本：V1.1.0

日期：2024-12-26

文件密级：□绝密 □秘密 □内部资料 ■公开

免责声明

本文档按"现状"提供，瑞芯微电子股份有限公司（"本公司"，下同）不对本文档的任何陈述、信息和内容的准确性、可靠性、完整性、适销性、特定目的性和非侵权性提供任何明示或暗示的声明或保证。本文档仅作为使用指导的参考。

由于产品版本升级或其他原因，本文档将可能在未经任何通知的情况下，不定期进行更新或修改。

商标声明

"Rockchip"、"瑞芯微"、"瑞芯"均为本公司的注册商标，归本公司所有。

本文档可能提及的其他所有注册商标或商标，由其各自拥有者所有。

版权所有 © 2024 瑞芯微电子股份有限公司

超越合理使用范畴，非经本公司书面许可，任何单位和个人不得擅自摘抄、复制本文档内容的部分或全部，并不得以任何形式传播。

瑞芯微电子股份有限公司

Rockchip Electronics Co., Ltd.

地址： 福建省福州市铜盘路软件园A区18号

网址： www.rock-chips.com

客户服务电话： +86-4007-700-590

客户服务传真： +86-591-83951833

客户服务邮箱： fae@rock-chips.com

前言

概述

产品版本

支持芯片	RT-Thread 版本
ALL	ALL

读者对象

本文档（本指南）主要适用于以下工程师：

软件开发工程师

修订记录

版本号	作者	修改日期	修改说明
V1.0.0	吴达超	2020-02-18	初始版本
V1.0.1	陈谋春	2020-03-17	修正链接
V1.0.2	陈谋春	2020-05-27	修正格式
V1.1.0	吴达超	2024-12-26	添加内容适配所有芯片

目录

Rockchip RT-Thread I2C开发向导

1. 功能配置
2. 板级配置
3. 代码和 API 使用
   - 3.1 代码位置
   - 3.2 I2C API 接口
     - 3.2.1 查找 I2C 总线设备
     - 3.2.2 I2C Write
     - 3.2.3 I2C Read
   - 3.3 开发指南
   - 3.4 I2C CMD 测试
4. Debug
   - 4.1 外设无响应
   - 4.2 Timeout

1. 功能配置

I2C 的功能配置分成两个部分：驱动框架和 VENDOR 配置，具体需要的配置如下：

```
# I2C驱动框架
RT_USING_I2C [=y]
# VENDOR驱动，根据需求启用你需要的I2C总线
RT_USING_I2C0 [=y]
RT_USING_I2C1 [=y]
RT_USING_I2C2 [=y]
```

2. 板级配置

- board.c

在板级配置文件里面 include 头文件，以及 I2C 总线配置表，去配置总线速率，目前支持到 100K，400K，1000K (配置速度时，请确认上升沿时间是符合该速度模式下的时序规范)，其它速率需要手动添加。板级配置举例：

```
#ifdef RT_USING_I2C
#include "drv_i2c.h"
#endif

#ifdef RT_USING_I2C
const struct rockchip_i2c_config rockchip_i2c_config_table[] =
{
#ifdef RT_USING_I2C0
    {
        .id = I2C0,
        .speed = I2C_400K,
    },
#endif
    { /* sentinel */ }
};
#endif
```

- iomux.c

I2C 的 IO 都是复用的，所以需要确保 IOMUX 的配置中相关的 IO 切到 I2C 模式，具体参见开发指南中的 IOMUX章节。

3. 代码和 API 使用

3.1 代码位置

- 控制器驱动层代码 ./bsp/rockchip/common/driver/i2c/drv_i2c.c
- 控制器 HAL 层代码 ./bsp/rockchip/common/hal/lib/hal/src/hal_i2c.c

3.2 I2C API 接口

3.2.1 查找 I2C 总线设备

```
#define I2C_BUS_NAME      "i2c1"        /* I2C总线设备名称 */
struct rt_i2c_bus_device *i2c_bus;      /* I2C总线设备句柄 */

/* 查找I2C总线设备，获取I2C总线设备句柄 */
i2c_bus = (struct rt_i2c_bus_device *)rt_device_find(name);
if (i2c_bus == RT_NULL)
  return -RT_ERROR;
```

3.2.2 I2C Write

```
static rt_err_t i2c_write(struct rt_i2c_bus_device *bus, rt_uint8_t addr,
 rt_uint8_t reg, rt_uint8_t *data)
{
    rt_uint32_t buf_size = 1;
    struct rt_i2c_msg msgs;
    rt_uint8_t buf[3];

    buf[0] = reg; //cmd
    if (data != RT_NULL)
    {
        buf[1] = data[0];
        buf[2] = data[1];
        buf_size = 3;
    }

    msgs.addr = addr;
    msgs.flags = RT_I2C_WR;
    msgs.buf = buf;
    msgs.len = buf_size;

    /* 调用I2C设备接口传输数据 */
    if (rt_i2c_transfer(bus, &msgs, 1) == 1)
    {
        return RT_EOK;
    }
    else
    {
        return -RT_ERROR;
    }
}
```

3.2.3 I2C Read

```
static rt_err_t i2c_read(struct rt_i2c_bus_device *bus,  rt_uint8_t addr,
 rt_uint8_t len, rt_uint8_t *buf)
{
    struct rt_i2c_msg msgs;

    msgs.addr = addr;
    msgs.flags = RT_I2C_RD;
    msgs.buf = buf;
    msgs.len = len;

    /* 调用I2C设备接口传输数据 */
    if (rt_i2c_transfer(bus, &msgs, 1) == 1)
    {
        return RT_EOK;
    }
    else
    {
        return -RT_ERROR;
    }
}
```

3.3 开发指南

I2C 驱动遵循 RTT 系统标准 I2C 驱动框架，因此可直接参考 RTT 官方I2C开发指南。

3.4 I2C CMD 测试

I2C CMD 代码 components/drivers/i2c/i2c_cmd.c

与 Linux i2c tools 用法一样，支持 CMD 包括：i2cdetect，i2cdump，i2cget，i2cset。例如 i2cdetect 的用法：

```
# i2cdetect -y 0
     0  1  2  3  4  5  6  7  8  9  a  b  c  d  e  f
00:          -- -- -- -- -- -- -- -- -- -- -- -- --
10: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
20: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
30: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
40: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
50: -- 51 -- -- -- -- -- -- -- -- -- -- -- -- -- --
60: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
70: -- -- -- -- -- -- -- --
```

4. Debug

4.1 外设无响应

返回值 HAL_NODEV = (-19)，表示外设无响应 NACK，

- 很可能设备地址错了，地址不包含读写位。
- I2C 外设器件的供电电压或者上电时序有问题，无法正常工作。

4.2 Timeout

常见导致 timeout 问题的原因：

- I2C 上拉电源没有，或者上拉电阻电压不对，与 IO 电压不匹配，如以及电压不够等。
- iomux 没有配置正常。
- 应用上重复使用了该引脚，造成工作异常。
- 外设拉低 I2C 总线。